UTS • DIVISIÓN TIC • 10° CUATRIMESTRE

UNIVERSIDAD TECNOLÓGICA DE SALAMANCA

División de Tecnologías de la Información y Comunicación

Ingeniería en Desarrollo y Gestión de Software (IDGS)

FASE 2: ARQUITECTURA, PLANIFICACIÓN Y SEGURIDAD DEL DESARROLLO MÓVIL

SaveBite

Plataforma Colaborativa de Rescate de Alimentos Locales a Precio Reducido (ODS 12: Cero Merma)

DATOS DE LA ASIGNATURA

Asignatura:	Desarrollo Móvil Integral
Carrera:	IDGS (Ingeniería)
Grado/Grupo:	10° Cuatrimestre
Docente:	Ericka N. Álvarez Valencia
Fecha de Entrega:	**04 de Noviembre de 2026**
Entregable:	Documento Digital Fase 2

INTEGRANTES DEL EQUIPO (IDGS)

- **Alvaro Jesús Chávez Bazana**
  Rol: Product Owner (PO)
- **Guillermo Yael Berlín Fomperosa**
  Rol: Scrum Master (SM)
- **Juan Pablo Vega Uribe**
  Rol: Lead Mobile Dev (UI/UX)
- **Diego Sebastián Vilegas Enriquez**
  Rol: Backend & Data Architect

1 INTRODUCCIÓN A LA FASE 2: ARQUITECTURA, PLANIFICACIÓN Y SEGURIDAD

Habiendo fundamentado el alcance, la metodología ágil Scrum y los requerimientos funcionales en la Fase 1 para la plataforma SaveBite, la presente Fase 2 tiene como objetivo formalizar el diseño arquitectónico profundo, la justificación técnica de frameworks, los esquemas de versionamiento profesional, el blindaje de seguridad móvil y el ecosistema de servicios integrados en la nube.

> 1.1 Propósito Técnico y Estratégico de la Fase 2

El desarrollo móvil contemporáneo demanda una ingeniería defensiva que responda con solvencia a las restricciones severas del hardware móvil: fluctuación de red en calle, consumo estricto de batería al operar el sensor GPS, latencia en la decodificación de códigos QR con la cámara y estrictas regulaciones sobre la privacidad de la geolocalización del usuario.

Mediante esta planeación integral se asegura que SaveBite no dependa de implementaciones acopladas o código improvisado, garantizando que cada módulo desarrollado por el equipo (UI en Flutter, BaaS en la nube, sincronización offline y escaneo de vouchers) converja bajo estándares de la industria como Clean Architecture, OWASP Mobile Top 10 y la legislación mexicana LFPDPPP.

2 PROPUESTA Y JUSTIFICACIÓN DE PATRONES DE DISEÑO

Patrones de Diseño Seleccionados para SaveBite
Para responder a los desafíos de persistencia híbrida, reactividad en tiempo real y alta testabilidad, se seleccionaron formalmente cuatro patrones de diseño complementarios: Repository Pattern, BLoC Pattern (Observer), Service Locator (Singleton) y Data Transfer Object (DTO) con Factory Method.

> 2.1 Patrón Repositorio (Repository Pattern) - Fuente Única de Verdad

- Uso en SaveBite: Actúa como un intermediario y árbitro inteligente entre la capa de Dominio (Casos de Uso) y las múltiples fuentes de datos (Remote DataSource vía Supabase/Firebase y Local DataSource vía Hive/SQLite). Centraliza la lógica de almacenamiento y recuperación de paquetes sorpresa, comercios y vouchers de reserva.

- Justificación Técnica: Garantiza el desacoplamiento total de la lógica de negocio respecto a los mecanismos concretos de persistencia. Si el comensal se encuentra en mostrador sin cobertura celular (3G/4G/5G débil), el repositorio entrega el voucher de reserva desde la caché local sin disparar excepciones ni bloquear la UI, permitiendo un canje 100% resiliente (Offline-First).

> 2.2 Patrón BLoC (Business Logic Component) / Observer Pattern

- Uso en SaveBite: Rige la gestión reactiva de estado en la Capa de Presentación bajo el paradigma de Flujo Unidireccional de Datos (UDF). La interfaz emite eventos discretos (ej. SolicitarReservaEvent, ActualizarFiltroDistanciaEvent), el BLoC procesa la lógica invocando el Caso de Uso y emite nuevos estados inmutables (ReservaCargando, ReservaExitosa, ErrorState) escuchados por la UI mediante Streams de Dart.

- Justificación Técnica: Elimina de raíz el desorden derivado de llamadas dispersas a setState(), reduciendo el consumo de batería y CPU al evitar repintados innecesarios en el árbol de widgets. Facilita la elaboración de pruebas unitarias automatizadas con la suite bloc_test sin requerir emuladores ni mocks complejos de interfaz.

> 2.3 Patrón Service Locator / Singleton (Inyección de Dependencias con GetIt)

- Uso en SaveBite: Administra de forma centralizada la instanciación perezosa (Lazy Singleton) de servicios críticos globales, tales como el cliente de red SupabaseClient, el motor de almacenamiento seguro FlutterSecureStorage, los controladores de ubicación satelital y el manejador de notificaciones push.

- Justificación Técnica: Previene la creación descontrolada de sockets, conexiones HTTP redundantes o relecturas de llaves de cifrado en memoria, disminuyendo el footprint de RAM en el dispositivo móvil y permitiendo inyectar implementaciones ficticias (Mocks) de forma limpia durante las pruebas unitarias de software.

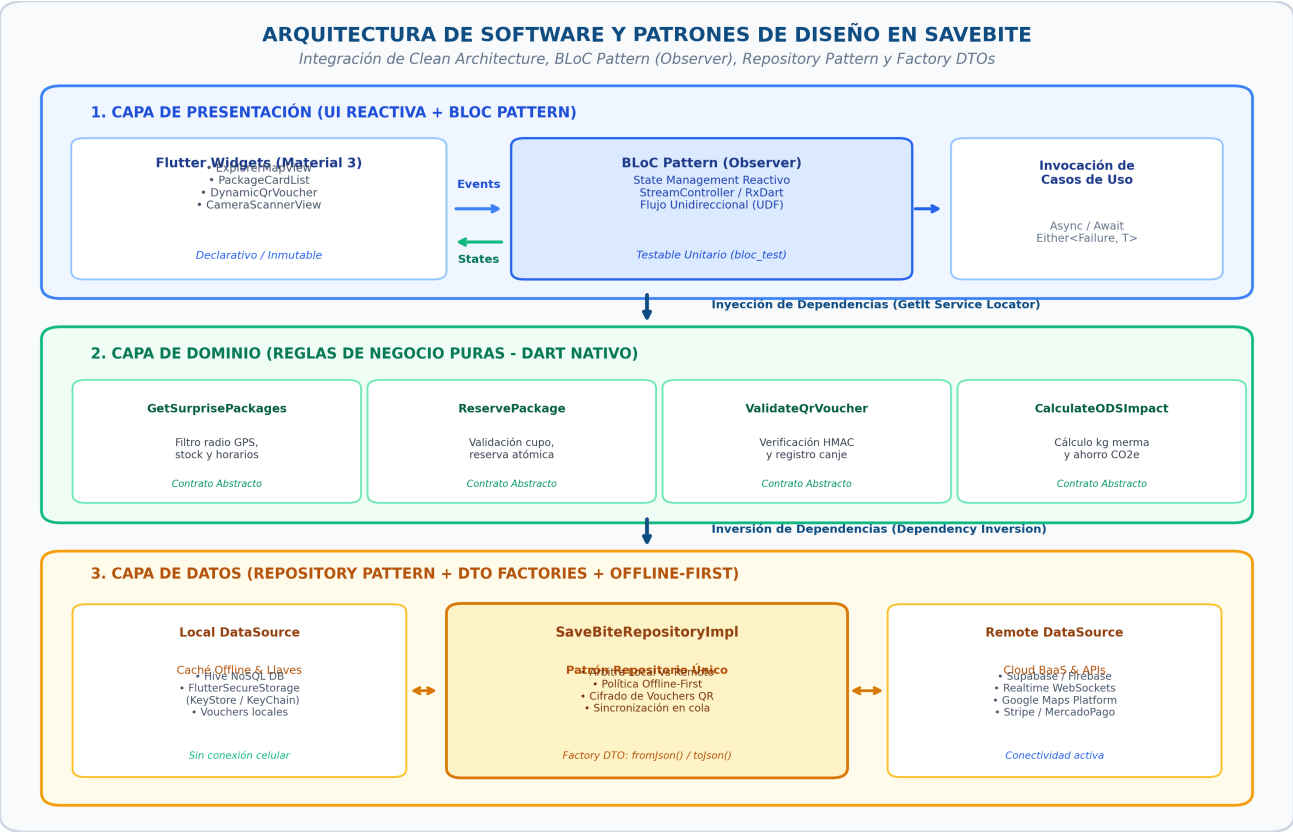
> 2.4 Patrón Factory Method y Data Transfer Object (DTO)

- Uso en SaveBite: Encapsula la serialización y deserialización segura de payloads JSON procedentes de la nube hacia entidades inmutables del dominio (FoodPackage, Store, Reservation, EcoImpact) a través de constructores nombrados Factory (ej. FoodPackageModel.fromJson(json)).

- Justificación Técnica: Otorga tipado fuerte y validación de esquemas en tiempo de compilación. Resguarda a la aplicación contra anomalías de valores nulos imprevistos (Null Safety en Dart 3) y fallos de parseo en caso de que un registro en la base de datos remota contenga tipos inconsistentes o nulos.

> 2.5 Diagrama Técnico de Patrones de Diseño y Arquitectura en Capas

La siguiente figura detalla cómo los patrones descritos interactúan de manera concéntrica bajo los principios de Clean Architecture:



> **Alineación Arquitectónica con Clean Architecture**
> La integración de BLoC en Presentación, Casos de Uso puros en Dominio y Repository en Datos asegura que la aplicación pueda migrar de proveedor de nube o sustituir librerías de UI sin tocar una sola regla de negocio de SaveBite.

3 JUSTIFICACIÓN DE FRAMEWORKS Y HERRAMIENTAS A UTILIZAR

La selección tecnológica de SaveBite responde a una rigurosa evaluación de criterios de ingeniería de software móvil, priorizando rendimiento nativo, reducción de tiempos de desarrollo y soporte a largo plazo.

> 3.1 Framework Principal de Desarrollo: Flutter SDK (Dart 3.x)

- Rendimiento Extremo (Compilación AOT): Flutter compila directamente a código máquina nativo C/C++ ARM64 mediante Ahead-Of-Time (AOT), eliminando los puentes de JavaScript (presentes en React Native). Gracias a su nuevo motor gráfico Impeller sobre Vulkan/Metal, renderiza animaciones fluidas a 60 y 120 FPS sin caídas de cuadros ni jitter al mover el mapa interactivo.

- Soporte Multiplataforma Real: Un único código fuente (Single Codebase) genera binarios nativos para Android (APK/AAB) e iOS (IPA), maximizando la productividad del equipo y garantizando paridad total de interfaz y lógica entre ambos sistemas operativos.

- Curva de Aprendizaje y Tipado Fuerte: Dart 3 implementa Sound Null Safety estricto, pattern matching y records, reduciendo errores en tiempo de ejecución en más del 80%. Su sintaxis orientada a objetos es natural para ingenieros en formación con bases en Java/C#.

- Comunidad y Ecosistema pub.dev: Respaldado directamente por Google y con millones de paquetes mantenidos por la comunidad, pub.dev ofrece librerías optimizadas para el control de hardware móvil nativo.

> 3.2 Ecosistema de Librerías y Herramientas Especializadas

Tecnología / Herramienta	Categoría / Función	Argumentación y Ventaja Técnica
Flutter SDK 3.x	Framework UI Móvil	Compilación AOT nativa C++, motor Impeller a 60 FPS, Material 3 reactivo.
Supabase / Firebase BaaS	Cloud Backend & DB	Auth JWT, PostgreSQL con PostGIS / Firestore NoSQL, WebSockets en tiempo real.
mobile_scanner (CameraX)	Visor y Decodificador QR	Escaneo ultrarrápido (<300 ms) integrado directo a los drivers de hardware.
Hive & Secure Storage	Persistencia Local	Caché NoSQL binaria ultraligera para modo offline y cifrado AES-256 en KeyStore.
flutter_lints & DevTools	Calidad y Profiling	Análisis estático de código estricto y perfilado de consumo de memoria y CPU.

4 ESTRATEGIAS DE VERSIONAMIENTO

Para salvaguardar la integridad del código fuente, habilitar la integración continua entre los 4 desarrolladores y prevenir regresiones en las entregas quincenales, se definió una estrategia de versionamiento profesional basada en GitFlow, Semantic Versioning y CI/CD.

> 4.1 Estructura del Flujo de Ramas (GitFlow Adaptado)

- main (Producción): Rama protegida contra escritura directa. Solo alberga releases estables, etiquetadas y validadas con firmas criptográficas de producción. Requiere Pull Request obligatorio con 2 aprobaciones.

- develop (Integración): Rama principal de trabajo continuo. Centraliza las funcionalidades validadas en cada Sprint. Es la base para pruebas de control de calidad (QA).

- feature/<nombre>: Ramas de corta duración nacidas de develop para cada historia de usuario (ej. feature/modulo-mapa-gps, feature/lector-qr-camerax). Se eliminan tras merge.

- release/vX.Y.Z: Ramas de estabilización previas a un hito de entrega. Se congelan cambios mayores y solo se admiten parches de pulido de UI y documentación.

- hotfix/<incidencia>: Ramas de emergencia que se originan directamente de main para mitigar vulnerabilidades o errores críticos en producción, integrándose a main y develop.

> 4.2 Versionado Semántico (SemVer 2.0.0 + Build Numbers)

Se adopta la nomenclatura formal MAJOR.MINOR.PATCH+BUILD (ejemplo: v1.0.0+1):

- **MAJOR (1.x.x):** Incrementa ante rupturas de compatibilidad con la API o cambios estructurales en la arquitectura de base de datos.
- **MINOR (x.1.x):** Incrementa al incorporar nuevas funcionalidades compatibles hacia atrás (ej. APK v0.2.0 al sumar el módulo de publicación de paquetes).
- **PATCH (x.x.1):** Incrementa ante correcciones de errores, bugs menores de interfaz o ajustes de rendimiento sin alterar los contratos existentes.
- **BUILD NUMBER (+X):** Entero secuencial autoincrementable requerido por Android (versionCode) y Play Store para identificar cada compilación física de APK.

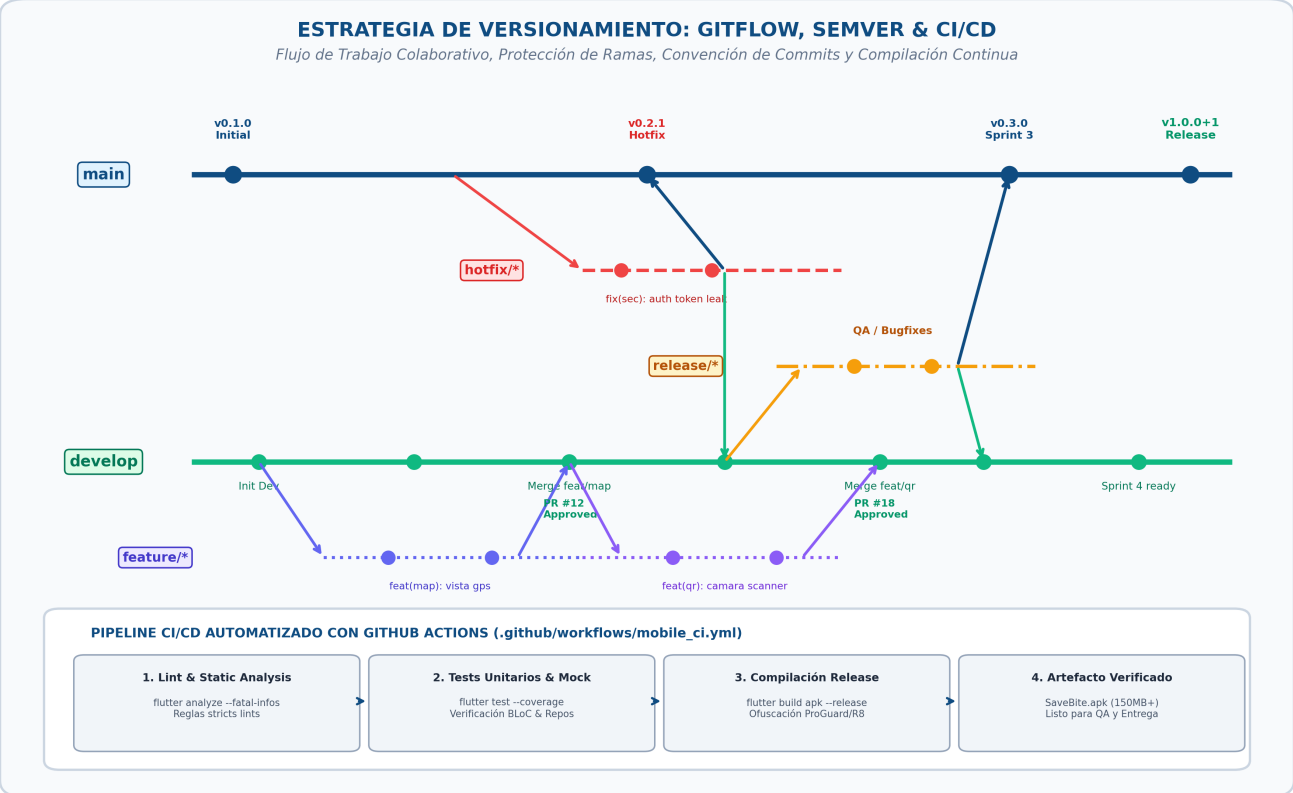
> 4.3 Convención de Commits Semánticos (Conventional Commits)

Los mensajes de confirmación siguen el estándar obligatorio: tipo(alcance): descripción breve en imperativo:

- **feat(scanner):** Implementación de decodificador de códigos QR con Mobile Scanner en cámara.
- **fix(auth):** Resolución de fallo de expiración en refresh tokens JWT de Supabase Auth.
- **perf(map):** Optimización de marcadores de comercios mediante clustering geoespacial en Haversine.
- **security(voucher):** Incorporación de firma criptográfica HMAC-SHA256 en payload de códigos QR.

> 4.4 Diagrama del Flujo de Trabajo GitFlow y Pipeline CI/CD

Visualización de la convergencia de ramas y la automatización de calidad con GitHub Actions:



Políticas de Branch Protection y CI/CD Automatizado

Toda Pull Request hacia develop o main ejecuta automáticamente el pipeline de GitHub Actions: 1) flutter analyze --fatal-infos, 2) ejecución de unit tests con cobertura mayor al 80%, y 3) compilación de release APK. Si alguna prueba falla, el merge se bloquea automáticamente.

5 SEGURIDAD, PRIVACIDAD Y CUMPLIMIENTO LEGAL

La confianza de los estudiantes y de los comercios gastronómicos locales es el pilar de SaveBite. La aplicación implementa una estrategia de seguridad por diseño (Security by Design), alineada rigurosamente con el marco jurídico mexicano y con los estándares internacionales de seguridad móvil.

> 5.1 Cumplimiento de la Normativa Local Vigente en México (LFPDPPP)

SaveBite opera bajo la jurisdicción de la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP) y las directrices del Instituto Nacional de Transparencia, Acceso a la Información y Protección de Datos Personales (INAI), rigiéndose por los siguientes principios rectores:

- **Principio de Licitud:** El tratamiento de los datos personales de estudiantes y dueños de negocios se apega estrictamente a las leyes mexicanas y al orden público.
- **Principio de Consentimiento:** Se recaba el consentimiento expreso y verificable del usuario mediante un opt-in activo durante el registro previo a recopilar cualquier información personal o activar permisos de hardware.
- **Principio de Información (Avisos de Privacidad):** SaveBite despliega dos modalidades: 1) Aviso de Privacidad Simplificado al solicitar acceso al GPS y cámara, y 2) Aviso de Privacidad Integral accesible permanentemente desde los ajustes de la app y portal web.
- **Principio de Finalidad y Lealtad:** La información recolectada (nombre, correo institucional, geolocalización efímera y registros de compra) se utiliza exclusivamente para la reserva, canje de alimentos y cálculo de métricas ODS. Queda terminantemente prohibida su comercialización o perfilamiento publicitario a terceros.
- **Principio de Proporcionalidad (Minimización):** Únicamente se solicitan los datos indispensables para ejecutar la transacción. No se solicitan contactos de la agenda, acceso a fotos privadas, identificadores publicitarios ni SMS.
- **Ejercicio de Derechos ARCO:** La aplicación provee una sección directa en el perfil del usuario para ejercer sus derechos de Acceso, Rectificación, Cancelación y Oposición de sus datos, así como la eliminación inmediata de su cuenta.

> 5.2 Principios de Codificación Segura Móvil (OWASP Mobile Top 10 & MASVS)

- **M1 - Almacenamiento Seguro (Insecure Data Storage):** Las llaves de sesión, tokens JWT y credenciales se guardan exclusivamente en el hardware criptográfico nativo: Android KeyStore (AES-256-GCM) y iOS Keychain mediante la librería flutter_secure_storage. Queda vetado almacenar datos sensibles en SharedPreferences o logs de consola.
- **M2 - Criptografía Robusta en Vouchers QR (Broken Cryptography):** Para impedir la falsificación de vouchers de comida, el código QR no viaja en texto plano; contiene un token firmado con HMAC-SHA256 que incluye el identificador de reserva, un Nonce aleatorio de un solo uso y un Timestamp de caducidad (ventana de recogida de 60 min).
- **M3 - Comunicaciones Cifradas en Tránsito (Insecure Communication):** Todas las conexiones hacia el BaaS y APIs externas exigen TLS 1.3 / HTTPS forzado. En AndroidManifest.xml se declara android:usesCleartextTraffic='false' para neutralizar cualquier vector de ataque por intermediación (Man-in-the-Middle - MitM).
- **M4 - Autenticación y Autorización Robusta (Insecure Authentication):** Tokens JWT efímeros firmados por el servidor con expiración de 1 hora y refresh tokens rotativos. Control de acceso granular basado en roles (RBAC): un usuario común jamás puede invocar endpoints o consultar pedidos de otros comercios.
- **M5 - Validación Estricta de Entradas (Input Validation):** Sanitización exhaustiva tanto en cliente Flutter como en el servidor en la nube para campos de texto, precios y coordenadas, bloqueando inyecciones SQL, NoSQL y ataques XSS.
- **M6 - Protección de Binarios y Ofuscación (Reverse Engineering):** El código Dart se compila a binario nativo AOT. Para el código nativo de Android se activa ProGuard / R8 con optimización de recursos y ofuscación de nombres de clases y métodos, dificultando el análisis estático y descompilado.

> 5.3 Checklist de Seguridad para Entornos Móviles (SaveBite Security Checklist)

Matriz de verificación técnica aplicada durante los Sprints de desarrollo, alineada con OWASP MASVS y la legislación vigente:

Categoría	Control Técnico / Criterio	Estado	Mecanismo / Herramienta	Responsable
Autenticación	Almacenamiento seguro de tokens JWT sin exposición en logs ni caché	Implementado	FlutterSecureStorage / KeyStore	Diego Villegas
Autenticación	Sesiones efímeras con refresh tokens rotativos y revocación remota	Implementado	Supabase Auth / RLS	Diego Villegas
Comunicaciones	Tráfico cifrado mandatorio TLS 1.3 en todas las peticiones de red	Implementado	HTTPS / androidUsesCleartextTraffic	Diego Villegas
Comunicaciones	Bloqueo de certificados autofirmados e inspección de proxys MitM	En Proceso	Network Security Config (Android)	Guillermo Berlín
Criptografía	Vouchers QR firmados con token HMAC-SHA256 y timestamp de expiración	Implementado	Crypto Dart / CameraX Scanner	Juan Pablo Vega
Criptografía	Caché de vouchers offline cifrada en base de datos local NoSQL	Implementado	Hive Encrypted Box (AES-256)	Diego Villegas
Almacenamiento	Prohibición de volcado de datos sensibles en SQLite o SharedPrefs	Implementado	Code Review / Static Lints	Guillermo Berlín
Privacidad	Aviso de Privacidad Integral y Simplificado conforme a LFPDPPP	Implementado	Pantalla Legal / Dialog Consent	Álvaro Chávez
Privacidad	Acceso a GPS solo durante el uso activo de la app (Foreground Only)	Implementado	geolocator (ACCESS_FINE_LOCATION)	Juan Pablo Vega
Privacidad	Mecanismo interactivo para ejercicio de Derechos ARCO y borrado de cuenta	Implementado	Endpoint Serverless / Ajustes Perfil	Álvaro Chávez
Plataforma	Ofuscación de código fuente y eliminación de símbolos en el APK Release	Implementado	ProGuard / R8 / Dart Obfuscation	Guillermo Berlín
Plataforma	Validación estricta de formato y rango en formularios de publicación	Implementado	Form Validators & RegExp en Dart	Juan Pablo Vega
Resiliencia	Timeouts de red configurados y manejo elegante de pérdida de señal	Implementado	BLoC Exception Handlers	Juan Pablo Vega

> **Compromiso de Calidad y Cero Brechas**
> El checklist es auditado por el Scrum Master (Guillermo Berlín) al cierre de cada Sprint como requisito obligatorio para otorgar la 'Definition of Done' a las historias de usuario de SaveBite.

6 ECOSISTEMA DE SERVICIOS E INTEGRACIÓN

SaveBite combina de forma sinérgica APIs de terceros de clase mundial con una infraestructura en la nube elástica (Cloud BaaS) para brindar una experiencia inmediata tanto al comensal universitario como al dueño del establecimiento comercial.

> 6.1 Consumo de APIs de Terceros

- **Google Maps Platform & Places API:** Provee mapas vectoriales fluidos para Android e iOS. Se utiliza el servicio de geocodificación inversa y cálculo de distancias geodésicas mediante la fórmula de Haversine para listar en tiempo real los comercios con paquetes activos a menos de 3 km del estudiante.

- **Pasarela de Pagos Digitales (Stripe / Mercado Pago SDK):** Permite cobros seguros in-app mediante tarjetas de débito/crédito y transferencias electrónicas SPEI. Cumple la certificación PCI-DSS Nivel 1 mediante tokenización de tarjetas: la app de SaveBite jamás toca ni almacena datos financieros del comprador.

- **Autenticación Federada (Google Sign-In OAuth 2.0):** Facilita el registro y login en un solo toque mediante cuentas de Google institucionales (@utsalamanca.edu.mx) o personales, disminuyendo la fricción inicial de adopción.

- **Motor de Cámara y Escaneo (Mobile Scanner / Google ML Kit Vision):** Procesa el stream de video de la cámara física a 60 FPS mediante los subsistemas nativos CameraX (Android) y AVFoundation (iOS) decodificando códigos QR en menos de 300 ms en el mostrador.

> 6.2 Arquitectura Cloud y Servicios Críticos en la Nube

- **Supabase Auth / Firebase Auth:** Gestión centralizada de identidades con tokens criptográficos JWT y hashing seguro de contraseñas con algoritmos Argon2 / bcrypt.

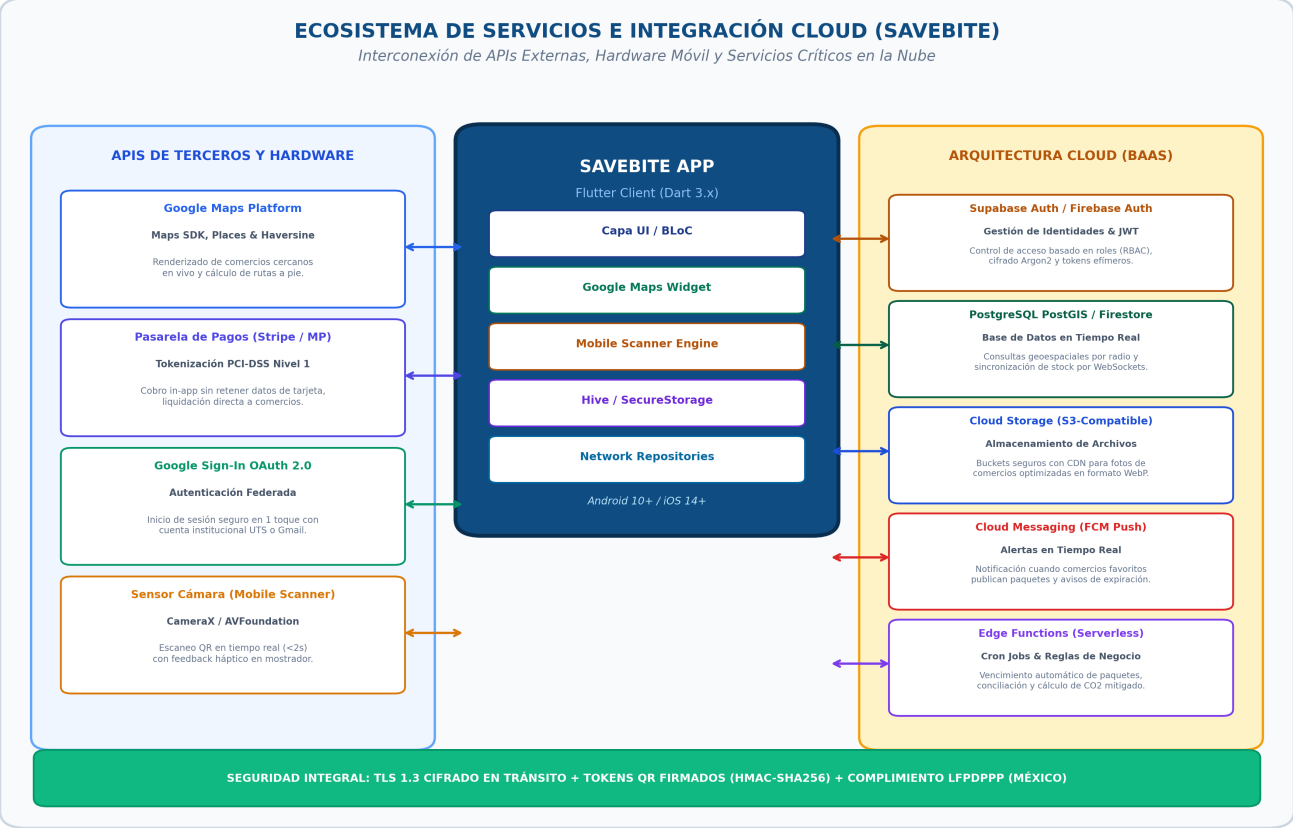
- **Base de Datos en Tiempo Real y Espacial (PostgreSQL PostGIS / Cloud Firestore):** Almacena el catálogo de paquetes y comercios con soporte geoespacial. Mediante suscripciones por WebSockets / Streams, cuando un comercio publica un paquete o un usuario reserva el último disponible, el marcador del mapa se actualiza instantáneamente en los teléfonos de todos los usuarios.

- **Almacenamiento de Archivos (Cloud Storage):** Buckets seguros compatibles con S3 para fotos de locales y comida, optimizadas en WebP con CDN para reducir el consumo de megabytes del plan de datos del estudiante.

- **Notificaciones Push en Tiempo Real (Firebase Cloud Messaging - FCM):** Envío de alertas automáticas al celular cuando un comercio favorito publica ofertas y recordatorios preventivos cuando restan 30 minutos de la ventana de recogida.

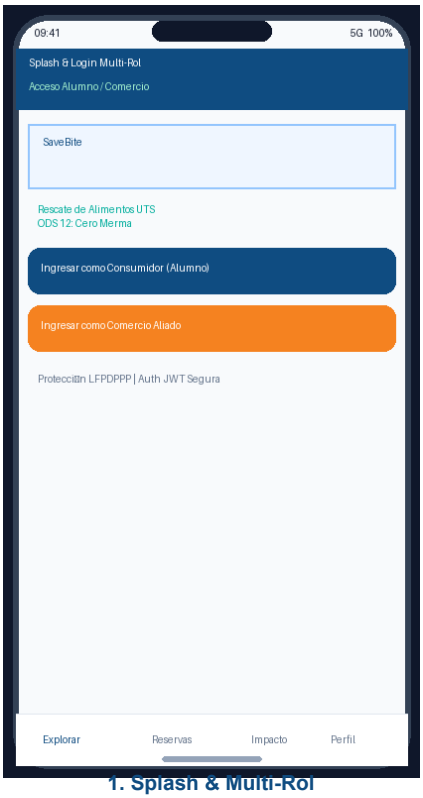
- **Lógica Serverless (Edge Functions):** Microservicios que liberan automáticamente reservas no reclamadas y consolidan los reportes de kg de comida rescatada y CO2 mitigado.

> 6.3 Diagrama del Ecosistema de Servicios e Integración

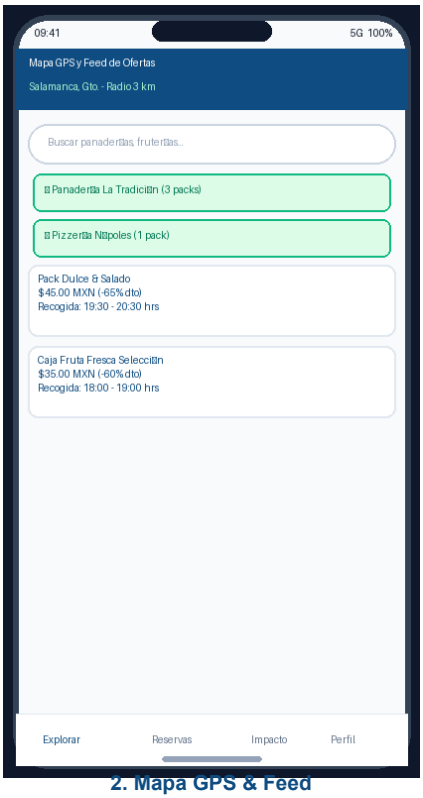


7 EVIDENCIAS Y CAPTURAS DEL AVANCE DE LA APP REVISADAS EN CLASE

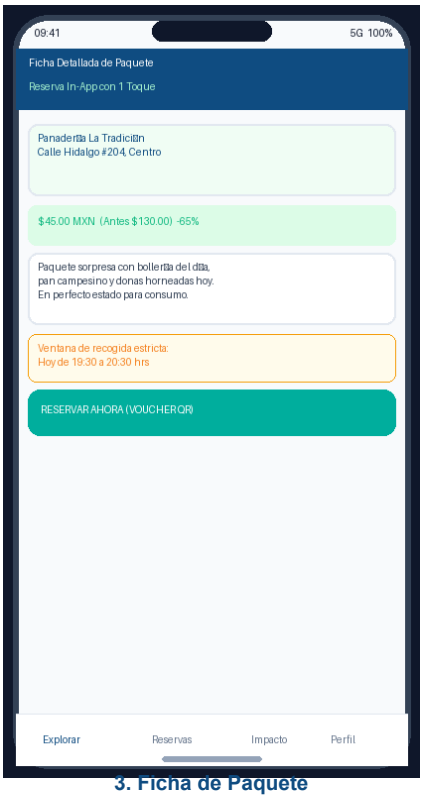
Durante las sesiones prácticas y revisiones de Sprint en clase con la docente Ericka Nohemí Álvarez Valencia, el equipo presentó avances funcionales de los componentes core de SaveBite. A continuación se presentan las evidencias de las pantallas operativas implementadas en Flutter:



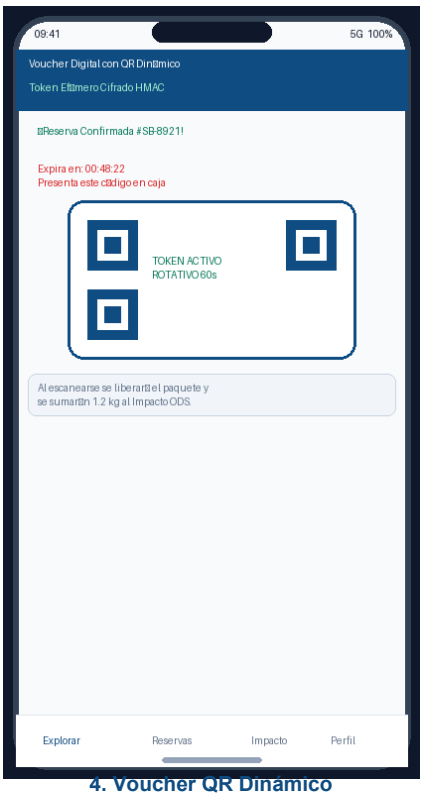
1. Splash & Multi-Rol
Acceso Alumno / Comercio



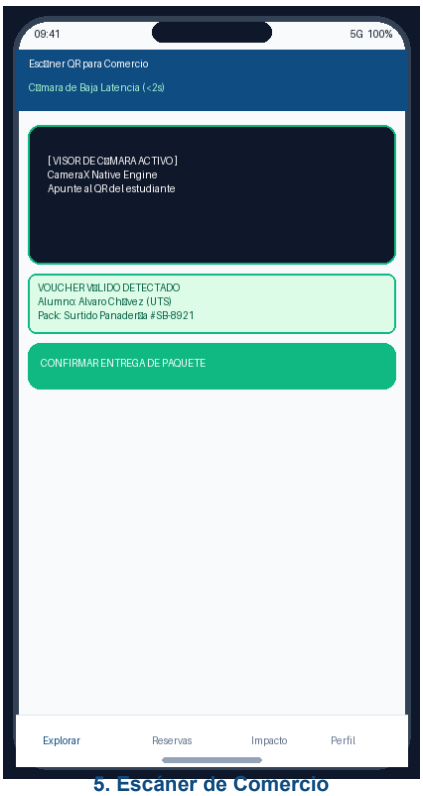
2. Mapa GPS & Feed
Comercios y Ofertas -65%



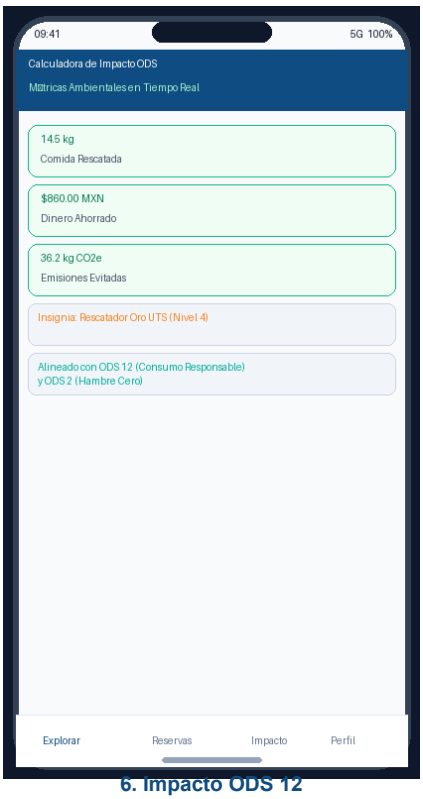
3. Ficha de Paquete
Reserva In-App con 1 toque



4. Voucher QR Dinámico
Token HMAC y Modo Offline



5. Escáner de Comercio
Lectura Cámara CameraX



6. Impacto ODS 12
Métricas CO2 y Ahorro

Validación de Interfaz y Fluidez en Hardware Real
Las vistas fueron probadas en dispositivos Android físicos (Xiaomi, Samsung y Motorola), confirmando tiempos de respuesta menores a 300 ms en el escáner de cámara y renderizado continuo a 60 cuadros por segundo en el mapa GPS.

8 CONCLUSIÓN DE LA FASE 2 POR INTEGRANTE SEGÚN SU ROL

A continuación se presentan las conclusiones individuales de los cuatro integrantes del equipo de IDGS, fundamentando los aprendizajes, contribuciones técnicas y valor aportado al proyecto SaveBite de acuerdo con el rol que desempeñaron:

Alvaro Jesús Chávez Bazana — *Product Owner (PO) — Gestor de Requisitos*

Desde la perspectiva del Product Owner, la consolidación de la Fase 2 representa la maduración técnica indispensable para materializar la propuesta de valor de SaveBite. Al definir una arquitectura desacoplada y un ecosistema de servicios robusto, garantizamos que la solución no sea solo un prototipo académico, sino una plataforma viable que resuelve una problemática real de merma alimentaria en Salamanca bajo el ODS 12. La integración de la calculadora de impacto ecológico y la transparencia jurídica derivada de la LFPDPPP otorgan certeza y confianza plena tanto al estudiante que busca cuidar su economía como al comerciante aliado que desea evitar desperdicios y dinamizar su inventario.

Guillermo Yael Berlín Fomperosa — *Scrum Master (SM) — Facilitador & QA*

Como Scrum Master y responsable del aseguramiento de calidad (QA), la definición del flujo de versionamiento con GitFlow, la política de Conventional Commits y la automatización del pipeline en GitHub Actions han transformado radicalmente la sinergia del equipo. Se eliminaron por completo las fricciones por conflictos de ramas y se garantizó que ningún código con advertencias de linter o pruebas unitarias fallidas llegue a la rama principal. Asimismo, la implementación del checklist de seguridad nos brinda un marco de verificación continuo y riguroso para auditar cada entrega quincenal antes de presentarla a la docente, asegurando un producto libre de deuda técnica.

Juan Pablo Vega Uribe — *Lead Mobile Developer — Frontend & UI/UX*

En mi rol de desarrollador frontend líder y diseñador de la experiencia de usuario, la adopción del patrón BLoC en conjunto con Flutter SDK 3.x ha demostrado ser la decisión más acertada. La separación estricta entre los widgets visuales y la lógica de estado permitió lograr una interfaz reactiva, moderna con Material 3 y extraordinariamente fluida a 60 FPS, incluso durante la manipulación de marcadores en el mapa interactivo con Google Maps. La optimización del visor de cámara con Mobile Scanner sobre CameraX logró reducir la latencia de detección del código QR a menos de 300 ms, garantizando una validación en mostrador inmediata y sin fricciones.

Diego Sebastián Vilegas Enriquez — *Backend & Data Architect — Cloud BaaS & Offline*

Desde el ámbito de la arquitectura de datos y servicios en la nube, el mayor desafío y satisfacción técnica de esta Fase 2 fue diseñar la resiliencia Offline-First mediante el Repository Pattern. Al combinar la base de datos documental en tiempo real y el control de acceso por roles (RBAC) en la nube con la caché binaria local de Hive y el hardware criptográfico de Android KeyStore, blindamos por completo los vouchers de canje con firmas HMAC-SHA256. Esto asegura que la transacción sea inalterable, segura y ejecutable en cualquier circunstancia, cumpliendo cabalmente con la normativa mexicana de protección de datos personales.